

```
# Basic libraries
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# ML utilities
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

# Advanced
from sklearn.model_selection import GridSearchCV
```

```
df = pd.read_csv('/content/Student Performance Dataset 2024.csv')

df.head()
```

	Student Id	Quiz	Midterm	Assignment_1	Assignment_2	Assignment_3	Project	Presentation	Final_Exam_Marks	Total
0	1	2.34	6.78	4.56	3.24	4.56	5.67	7.89	24.0	59
1	2	8.17	12.43	2.87	1.87	2.89	2.34	2.34	29.0	61
2	3	5.92	3.21	0.45	4.56	0.76	8.90	5.67	30.0	59
3	4	1.45	9.87	3.12	2.34	3.01	1.23	1.23	33.0	55
4	5	9.78	5.34	0.98	0.98	1.98	9.45	9.01	31.0	68

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 560 entries, 0 to 559
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Student Id            560 non-null   int64
1   Quiz                  560 non-null   float64
2   Midterm               560 non-null   float64
3   Assignment_1          560 non-null   float64
4   Assignment_2          560 non-null   float64
5   Assignment_3          560 non-null   float64
6   Project               560 non-null   float64
7   Presentation          560 non-null   float64
8   Final_Exam_Marks      560 non-null   float64
9   Total                 560 non-null   float64
10  Grade                 560 non-null   object
11  Categories             560 non-null   object
dtypes: float64(9), int64(1), object(2)
memory usage: 52.6+ KB
```

df.describe()

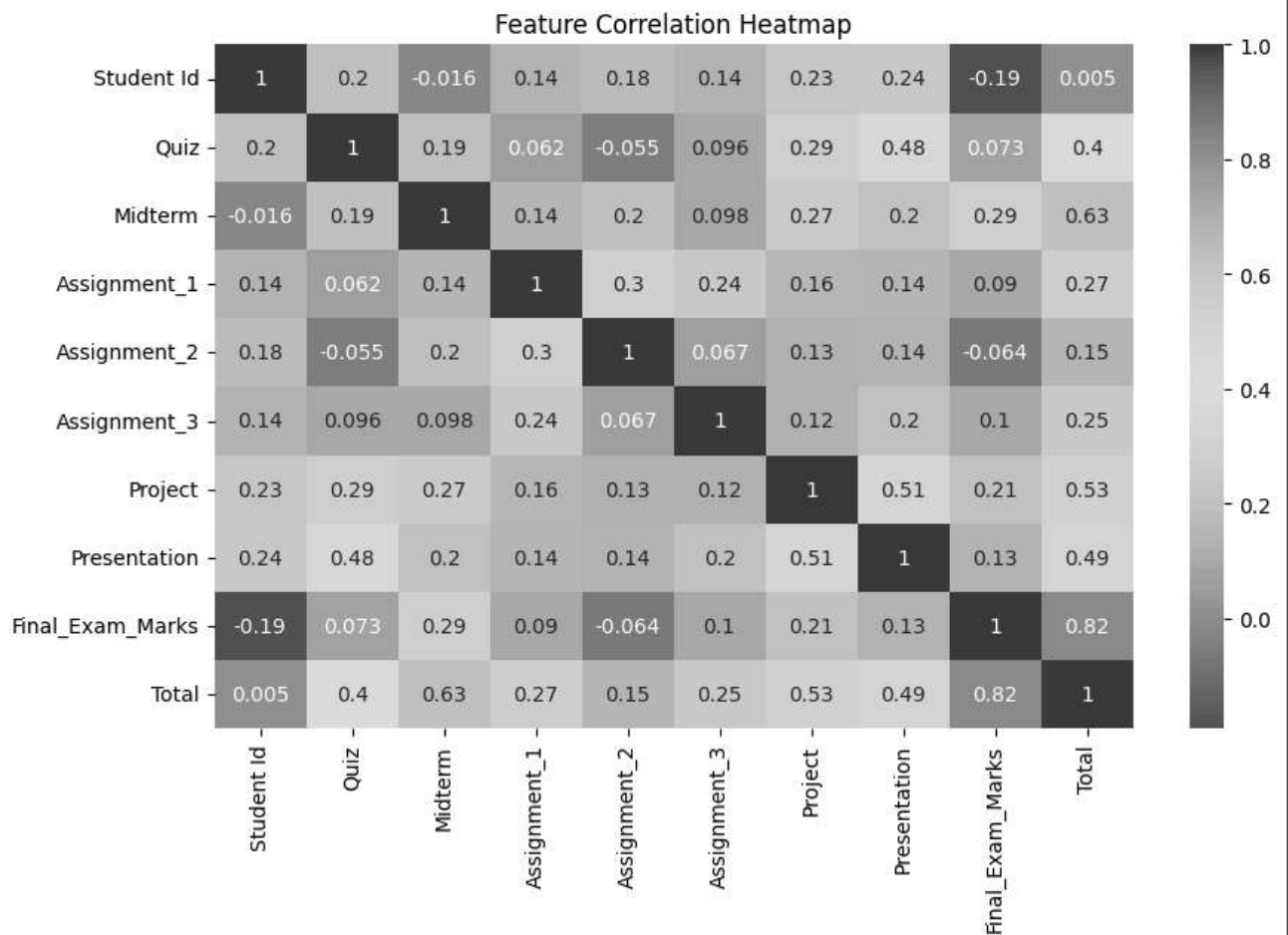
	Student Id	Quiz	Midterm	Assignment_1	Assignment_2	Assignment_3	Project	Presentation	Final Exam Marks
count	560.000000	560.000000	560.000000	560.000000	560.000000	560.000000	560.000000	560.000000	560.000000
mean	280.500000	6.481375	8.545743	3.492179	3.674073	3.505732	7.034429	7.294018	7.294018
std	161.802349	2.127193	3.740173	0.912075	0.907100	0.876958	1.913305	1.818343	1.818343
min	1.000000	0.000000	0.120000	0.090000	0.210000	0.120000	0.210000	0.210000	0.210000
25%	140.750000	5.120000	5.450000	2.980000	3.230000	3.010000	5.560000	6.010000	6.010000
50%	280.500000	6.560000	8.065000	3.560000	3.980000	3.630000	7.230000	7.560000	7.560000
75%	420.250000	8.230000	12.230000	4.230000	4.232500	4.120000	8.752500	8.900000	8.900000
max	560.000000	9.980000	15.000000	4.980000	4.980000	4.980000	9.980000	9.980000	9.980000

df.isnull().sum()

	0
Student Id	0
Quiz	0
Midterm	0
Assignment_1	0
Assignment_2	0
Assignment_3	0
Project	0
Presentation	0
Final_Exam_Marks	0
Total	0
Grade	0
Categories	0

dtype: int64

```
plt.figure(figsize=(10,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()
```



Data Preprocessing

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
def performance_label(score):
    if score >= 75:
        return "Good"
    elif score >= 50:
        return "Average"
    else:
        return "Poor"

df['Performance'] = df['Total'].apply(performance_label)
```

```
le = LabelEncoder()
df['Performance_Label'] = le.fit_transform(df['Performance'])
```

```
X = df.drop([
    'Student Id',
    'Total',
    'Final_Exam_Marks',
    'Grade',
    'Categories',
    'Performance',
    'Performance_Label'
], axis=1)
```

```
y = df['Performance_Label']
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42  
)
```

Model Building (Basic → Advanced)

```
lr = LogisticRegression(max_iter=1000)  
lr.fit(X_train, y_train)  
  
y_pred_lr = lr.predict(X_test)  
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
```

Logistic Regression Accuracy: 0.8125

```
dt = DecisionTreeClassifier(random_state=42)  
dt.fit(X_train, y_train)  
  
y_pred_dt = dt.predict(X_test)  
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

Decision Tree Accuracy: 0.7678571428571429

```
rf = RandomForestClassifier(n_estimators=200, random_state=42)  
rf.fit(X_train, y_train)  
  
y_pred_rf = rf.predict(X_test)  
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Random Forest Accuracy: 0.875

```
svm = SVC(kernel='rbf', probability=True)  
svm.fit(X_train, y_train)  
  
y_pred_svm = svm.predict(X_test)  
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
```

SVM Accuracy: 0.8214285714285714

```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)  
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)  
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.30)  
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
from xgboost import XGBClassifier
```

```
# XGBoost Model  
xgb = XGBClassifier(  
    n_estimators=200,
```

```

        learning_rate=0.1,
        max_depth=5,
        random_state=42,
        use_label_encoder=False,
        eval_metric='mlogloss'
    )

xgb.fit(X_train, y_train)

```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:16] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```

XGBClassifier(
    base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=None, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric='mlogloss',
    feature_types=None, feature_weights=None, gamma=None,
    grow_policy=None, importance_type=None,
    interaction_constraints=None, learning_rate=0.1, max_bin=None,
    max_cat_threshold=None, max_cat_to_onehot=None,
    max_delta_step=None, max_depth=5, max_leaves=None,
    min_child_weight=None, missing=nan, monotone_constraints=None,
    multi_strategy=None, n_estimators=200, n_jobs=None,
    num_parallel_tree=None, ...)

```

```

# Train accuracy
train_acc_xgb = accuracy_score(y_train, xgb.predict(X_train))

# Test accuracy
test_acc_xgb = accuracy_score(y_test, xgb.predict(X_test))

# Cross-validation
cv_acc_xgb = cross_val_score(xgb, X_scaled, y, cv=5).mean()

print(" XGBoost Training Accuracy :", round(train_acc_xgb * 100, 2), "%")
print(" XGBoost Test Accuracy      :", round(test_acc_xgb * 100, 2), "%")
print(" XGBoost CV Accuracy         :", round(cv_acc_xgb * 100, 2), "%")

```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:16] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:17] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:18] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:21] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:05:22] WARNING: /__w/xgboost/ Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
XGBoost Training Accuracy : 100.0 %
```

```
XGBoost Test Accuracy      : 81.25 %
```

```
XGBoost CV Accuracy        : 79.46 %
```

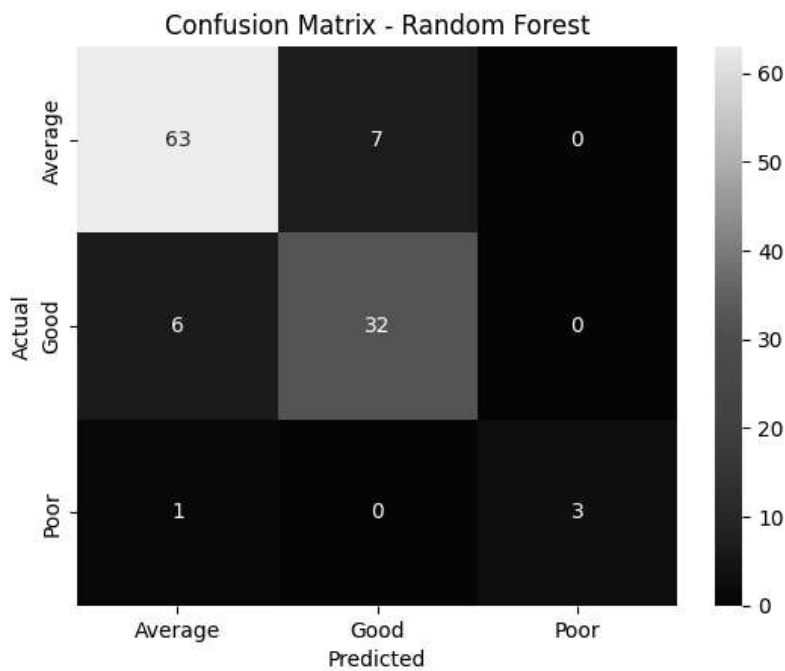
```
from sklearn.metrics import classification_report
```

```
y_pred = rf.predict(X_test)
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.90	0.90	0.90	70
1	0.82	0.84	0.83	38
2	1.00	0.75	0.86	4
accuracy			0.88	112
macro avg	0.91	0.83	0.86	112
weighted avg	0.88	0.88	0.88	112

```
cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=le.classes_,
            yticklabels=le.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



```
cv_scores = cross_val_score(rf, X_scaled, y, cv=5)
print("Cross Validation Accuracy:", cv_scores.mean())
```

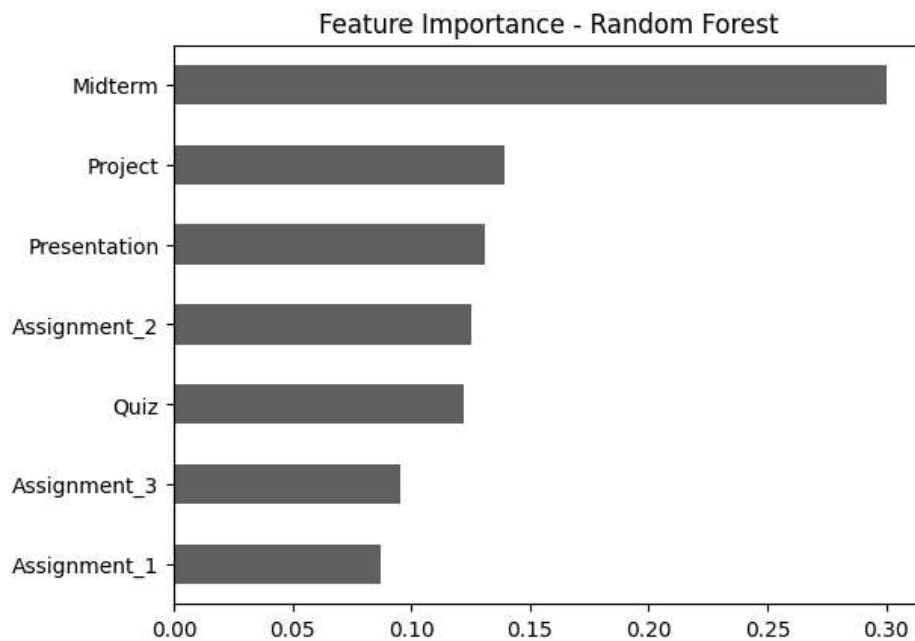
Cross Validation Accuracy: 0.8

```
#feature importance graph
import matplotlib.pyplot as plt
import pandas as pd

importances = rf.feature_importances_
features = X.columns

feat_imp = pd.Series(importances, index=features)
feat_imp.sort_values().plot(kind='barh')

plt.title("Feature Importance - Random Forest")
plt.show()
```



```
# Logistic Regression
test_acc_lr = accuracy_score(y_test, lr.predict(X_test))

# Decision Tree
test_acc_dt = accuracy_score(y_test, dt.predict(X_test))

# Random Forest
test_acc_rf = accuracy_score(y_test, rf.predict(X_test))

# SVM
test_acc_svm = accuracy_score(y_test, svm.predict(X_test))

# XGBoost
test_acc_xgb = accuracy_score(y_test, xgb.predict(X_test))
```

```
import matplotlib.pyplot as plt

# ♦ Model Names
models = [
    "Logistic Regression",
    "Decision Tree",
    "Random Forest",
    "SVM",
    "XGBoost"
]

# ♦ Apni Test Accuracies yaha daal
accuracies = [
    test_acc_lr * 100, # Logistic Regression
    test_acc_dt * 100, # Decision Tree
    test_acc_rf * 100, # Random Forest
    test_acc_svm * 100, # SVM
    test_acc_xgb * 100 # XGBoost
]

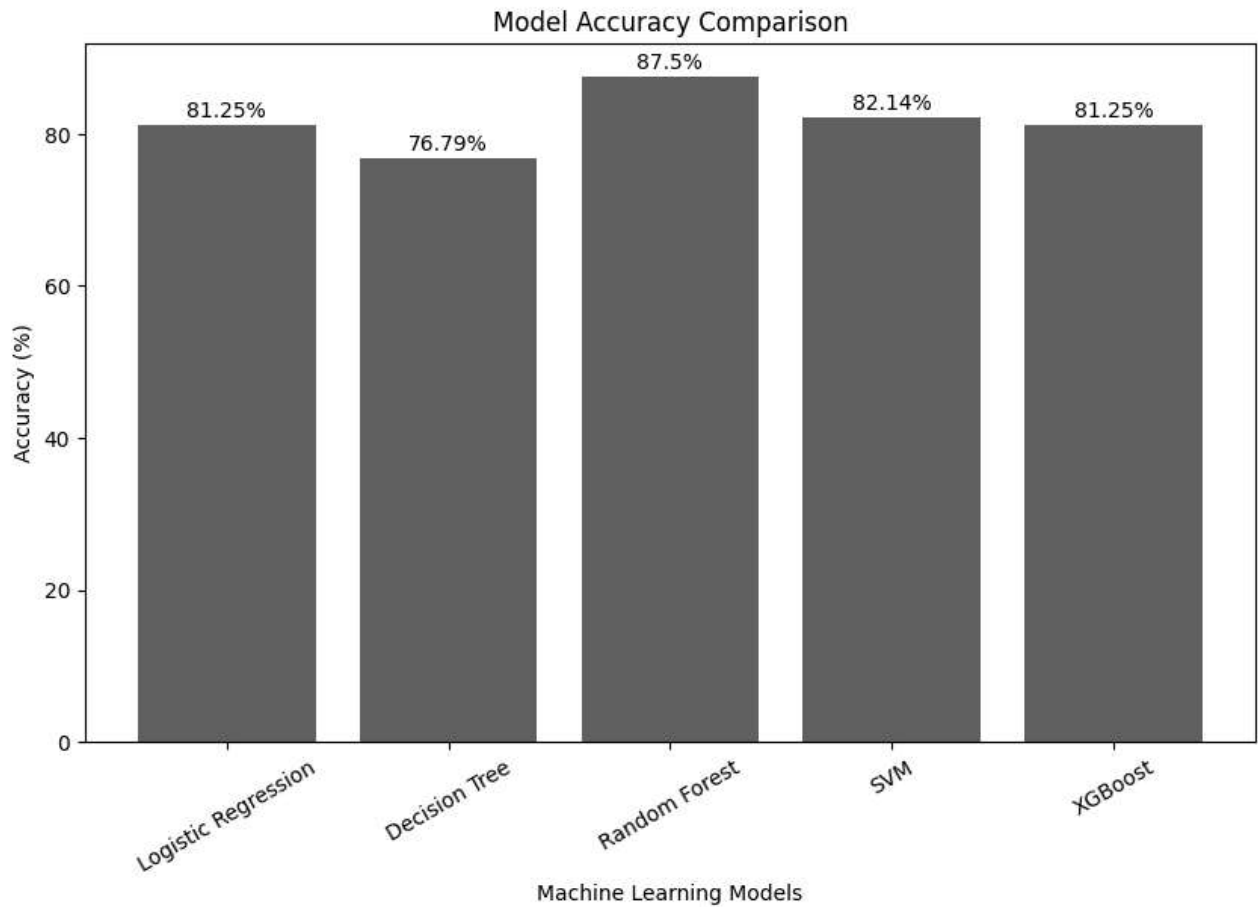
# ♦ Graph
plt.figure(figsize=(10,6))
plt.bar(models, accuracies)

plt.title("Model Accuracy Comparison")
plt.xlabel("Machine Learning Models")
```

```
plt.ylabel("Accuracy (%)")
plt.xticks(rotation=30)

# Value labels on bars
for i, v in enumerate(accuracies):
    plt.text(i, v + 1, f"{round(v,2)}%", ha='center')

plt.show()
```



```
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid,
                    cv=3,
                    scoring='accuracy')

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

Best Parameters: {'max_depth': None, 'n_estimators': 200}

```
def predict_student_performance(input_df):
    input_scaled = scaler.transform(input_df)

    pred = rf.predict(input_scaled)[0]
    prob = rf.predict_proba(input_scaled)[0]

    return {
```



```

        "Predicted Performance": le.inverse_transform([pred])[0],
        "Confidence Scores": {
            le.classes_[i]: float(round(prob[i], 2))    # convert to float
            for i in range(len(prob))
        }
    }
}

```

```

sample_student = X.iloc[[0]]

predict_student_performance(sample_student)

```

```

{'Predicted Performance': 'Average',
 'Confidence Scores': {'Average': 0.98, 'Good': 0.02, 'Poor': 0.0}}

```

```

# Model
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

# Train accuracy
train_acc = accuracy_score(y_train, rf.predict(X_train))

# Test accuracy
test_acc = accuracy_score(y_test, rf.predict(X_test))

# Cross-validation
cv_acc = cross_val_score(rf, X_scaled, y, cv=5).mean()

print("Training Accuracy      :", round(train_acc * 100, 2), "%")
print("Test Accuracy         :", round(test_acc * 100, 2), "%")
print("Cross-Validation Accuracy:", round(cv_acc * 100, 2), "%")

```

```

Training Accuracy      : 100.0 %
Test Accuracy         : 87.5 %
Cross-Validation Accuracy: 80.0 %

```

```
!pip install gradio
```

```

Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.12.0)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.13.0)
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.115.2)
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.0)
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.14.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.33.5)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.2)
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.9)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (26.2)
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.2.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.12.3)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.20)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.15.16)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.6)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.40.0)
Requirement already satisfied: tomkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.12.0)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.0)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.12.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.49.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (2025.3.0)
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (13.1)

```

```

Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0,>=3.0->grad)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->grad)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->grad)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<2.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<2.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->grad)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0)

```

```
import gradio as gr
```

```

def predict_by_student_id(student_id):

    # Check if ID exists
    if student_id not in df['Student Id'].values:
        return " Student ID not found"

    # Get student row
    student_row = df[df['Student Id'] == student_id]

    # Drop non-feature columns (same as training)
    student_features = student_row.drop([
        'Student Id',
        'Total',
        'Final_Exam_Marks',
        'Grade',
        'Categories',
        'Performance',
        'Performance_Label'
    ], axis=1)

    # Scale
    student_scaled = scaler.transform(student_features)

    # Predict
    pred = rf.predict(student_scaled)[0]
    performance = le.inverse_transform([pred])[0]

    return f" Predicted Performance: {performance}"

```

```

interface = gr.Interface(
    fn=predict_by_student_id,
    inputs=gr.Number(label="Enter Student ID"),
    outputs=gr.Textbox(label="Performance Prediction"),
    title="Educational Performance Forecasting",
    description="Enter Student ID to predict performance (Good / Average / Poor)"
)

interface.launch()

```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically set

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://2a0dc7077232209cde.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal

Educational Performance Forecasting

Enter Student ID to predict performance (Good / Average / Poor)

Enter Student ID

19

Performance Prediction

Predicted Performance: Average

Clear

Submit

Flag

Use via API  · Built with Gradio  · Settings 

Next steps: [Deploy to Cloud Run](#)

```
#save model
import joblib

joblib.dump(rf, "student_performance_model.pkl")

['student_performance_model.pkl']
```

```
model = joblib.load("student_performance_model.pkl")
```